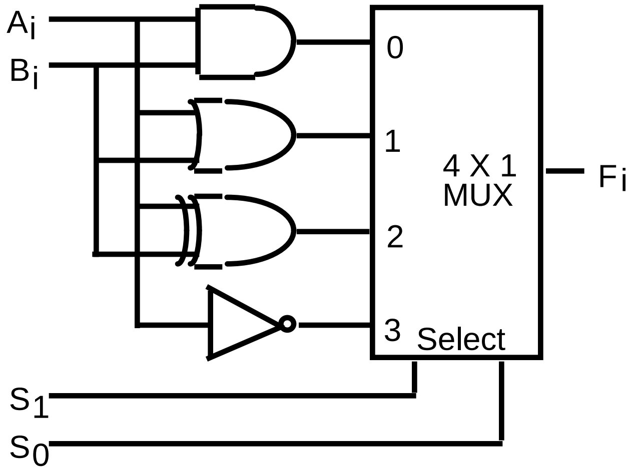
HARDWARE IMPLEMENTATION OF LOGIC MICROOPERATIONS



Function table

S_1 S_0	Output	μ-operation
0 0	$F = A \wedge B$	AND
0 1	$F = A \vee B$	OR
1 0	$F = A \oplus B$	XOR
1 1	$F = A'$	Complement

APPLICATIONS OF LOGIC MICROOPERATIONS

- Logic microoperations can be used to manipulate individual bits or a portions of a word in a register

- Consider the data in a register A. In another register, B, is bit data that will be used to modify the contents of A

• Selective-set	$A \leftarrow A \vee B$
• Selective-complement	$A \leftarrow A \oplus B$
• Selective-clear	$A \leftarrow A \wedge B'$
• Mask (Delete)	$A \leftarrow A \wedge B$
• Clear	$A \leftarrow A \oplus B$
• Insert	$A \leftarrow (A \wedge B) \vee C$
• Compare	$A \leftarrow A \oplus B$

SELECTIVE SET

- In a selective set operation, the bit pattern in B is used to *set* certain bits in A

$$
\begin{array}{ll}
1\,1\,0\,0 & A_t \\
\underline{1\,0\,1\,0} & \underline{B} \\
\hline
1\,1\,1\,0 & A_{t+1} \qquad (A \leftarrow A \vee B)
\end{array}
$$

- If a bit in B is set to 1, that same position in A gets set to 1, otherwise that bit in A keeps its previous value

SELECTIVE COMPLEMENT

- In a selective complement operation, the bit pattern in B is used to *complement* certain bits in A

$$\begin{array}{ll} 1\,1\,0\,0 & A_t \\ 1\,0\,1\,0 & B \\ \hline 0\,1\,1\,0 & A_{t+1} \end{array} \qquad (A \leftarrow A \oplus B)$$

- If a bit in B is set to 1, that same position in A gets complemented from its original value, otherwise it is unchanged

SELECTIVE CLEAR

- In a selective clear operation, the bit pattern in B is used to *clear* certain bits in A

$$\begin{array}{ll} 1\,1\,0\,0 & A_t \\ 1\,0\,1\,0 & B \\ \hline 0\,1\,0\,0 & A_{t+1} \qquad (A \leftarrow A \wedge B') \end{array}$$

- If a bit in B is set to 1, that same position in A gets set to 0, otherwise it is unchanged

MASK OPERATION

- In a mask operation, the bit pattern in B is used to *clear* certain bits in A

$$\begin{array}{ll} 1\,1\,0\,0 & A_t \\ 1\,0\,1\,0 & B \\ \hline 1\,0\,0\,0 & A_{t+1} \end{array} \qquad (A \leftarrow A \wedge B)$$

- If a bit in B is set to 0, that same position in A gets set to 0, otherwise it is unchanged

CLEAR OPERATION

- The clear operation compares the words in A and B and produces an all 0’s result if the two numbers are equal.
- The result is achieved by an exclusive-OR microoperation.

$$
\begin{array}{ll}
1\,1\,0\,0 & A_t \\
1\,0\,1\,0 & B \\
\hline
0\,1\,1\,0 & A_{t+1} \qquad (A \leftarrow A \oplus B)
\end{array}
$$

- When A and B are equal, the two corresponding bits are either both 0 or 1. In either case exclusive-OR operation produces a 0. The all-0’s result is then checked to determine if the two numbers are equal.

INSERT OPERATION

- An insert operation is used to introduce a specific bit pattern into A register, leaving the other bit positions unchanged
- This is done as
  - A mask operation to clear the desired bit positions, followed by
  - An OR operation to introduce the new bits into the desired positions
  - Example
    - Suppose you wanted to introduce 1010 into the low order four bits of A:

      1101 1000 1011 0001 A (Original)

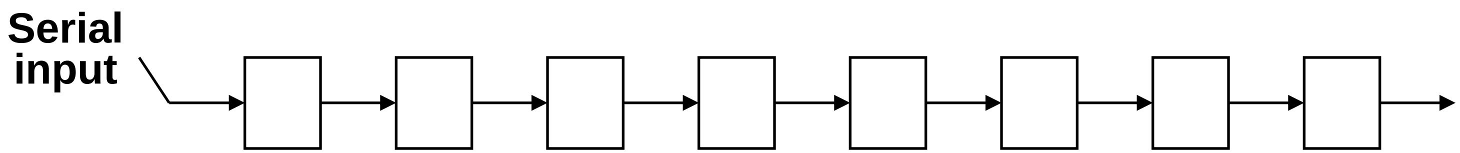
      1101 1000 1011 1010 A (Desired)

    -
```
1101 1000 1011 0001          A (Original)
1111 1111 1111 0000          Mask
--------------------------------------------
1101 1000 1011 0000          A (Intermediate)
0000 0000 0000 1010          Added bits
--------------------------------------------
1101 1000 1011 1010          A (Desired)
```

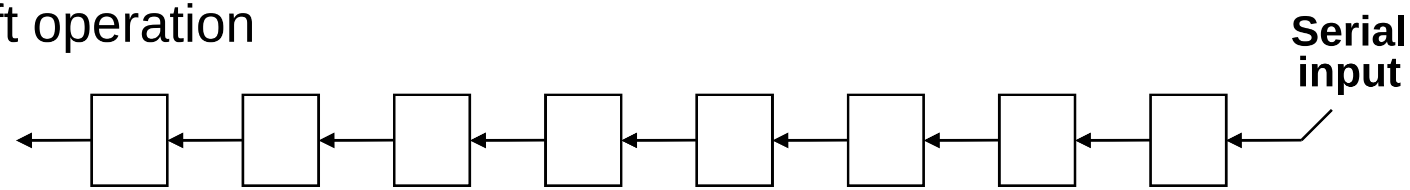
SHIFT MICROOPERATIONS

- There are three types of shifts
  - *Logical shift*
  - *Circular shift*
  - *Arithmetic shift*
- What differentiates them is the information that goes into the serial input

- A right shift operation



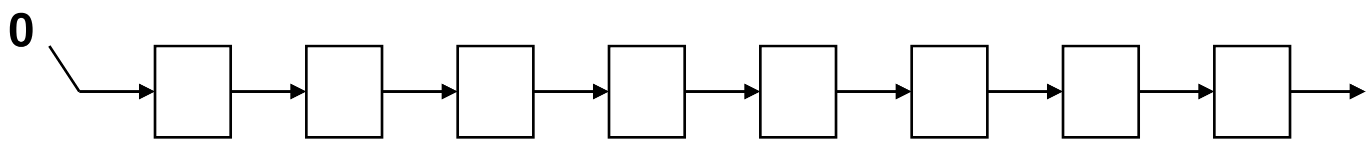
- A left shift operation



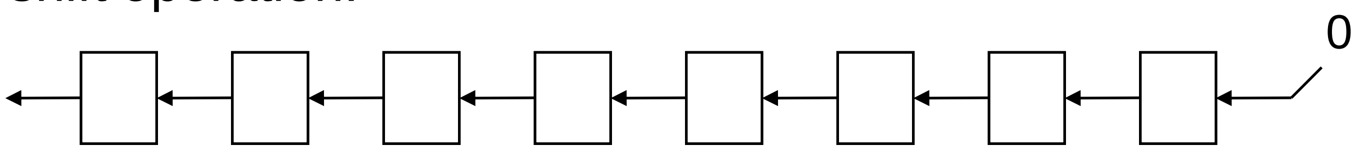
LOGICAL SHIFT

- In a logical shift the serial input to the shift is a 0.

- A right logical shift operation:



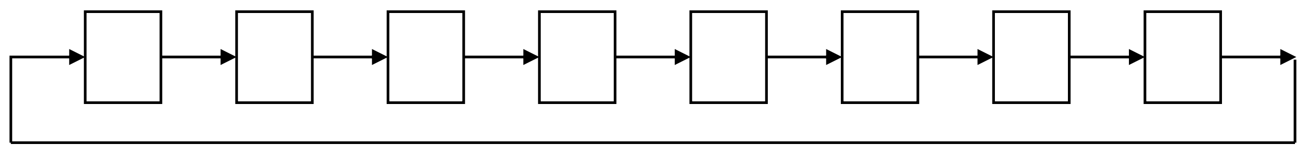
- A left logical shift operation:



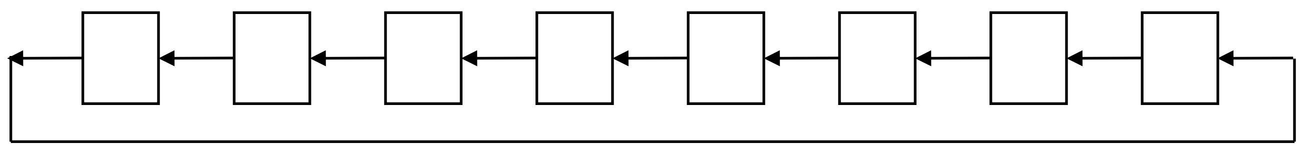
- In a Register Transfer Language, the following notation is used
  - *shl* for a logical shift left
  - *shr* for a logical shift right
  - Examples:
    - R2 ← *shr* R2
    - R3 ← *shl* R3

CIRCULAR SHIFT

- In a circular shift (also known as rotate operation) the serial input is the bit that is shifted out of the other end of the register.
- A right circular shift operation:



- A left circular shift operation:



- In a RTL, the following notation is used
  - *cil* for a circular shift left
  - *cir* for a circular shift right
  - Examples:
    - R2 ← *cir* R2
    - R3 ← *cil* R3